Neural Computing and Applications
https://doi.org/10.1007/s00521-021-06665-5

S. I. : EFFECTIVE AND EFFICIENT DEEP LEARNING

A deep learning-based resource usage prediction model for resource provisioning in an autonomic cloud computing environment

Mahfoudh Saeed Al-Asaly[1] · Mohamed A. Bencherif[2] · Ahmed Alsanad[1] · Mohammad Mehedi Hassan[1]

Received: 9 January 2021 / Accepted: 27 October 2021
© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2021

Abstract
Cloud computing enables clients to acquire cloud resources dynamically and on demand for their cloud applications and services. For cloud providers, especially, Software as a Service (SaaS) providers, the prediction of future cloud resource requirements, such as CPU usage for their cloud applications, to implement client requests is a complex task because it depends on incoming workloads. Due to workload fluctuations, it is difficult for SaaS cloud providers to predict or forecast future demand for resource usage in the next time interval and, accordingly, to allocate the required resources. Furthermore, cloud computing systems consist of many virtual machines (VMs), which increases the complexity of the prediction problem due to the correlations that exist between the large workload data in these VMs. Therefore, accurate resource usage forecasting remains a challenge, and relatively few studies have explored the prediction of CPU usage for VMs in cloud data centers. This paper proposes an autonomic and intelligent workload forecasting method for cloud resource provisioning based on the concept of autonomic computing and a deep learning approach. In particular, to predict future demand for CPU usage and determine how to respond to workload fluctuations in the next interval, we propose an efficient deep learning model based on a diffusion convolutional recurrent neural network (DCRNN). Existing deep learning models that are widely applied cannot handle accurate real-time forecasting due to the presence of inconsistent and nonlinear workloads in cloud computing systems. The goal of the proposed deep learning model is to improve forecasting accuracy and minimize the error between the predicted and the actual workloads. The effectiveness of the proposed DCRNN-based deep learning model was evaluated using experiments on a real-world dataset of PlanetLab's CPU usage traces. The results indicate that the proposed approach outperformed other existing deep learning models, achieving a mean absolute percentage error of 0.18 and root-mean-square error of 2.40.

Keywords Autonomic computing · Control MAPE loop · Deep learning · Diffusion convolutional recurrent neural network · Rang scaling (P100),Virtual machines · Z-score scaling (MS)

✉ Mohammad Mehedi Hassan
mmhassan@ksu.edu.sa

Mahfoudh Saeed Al-Asaly
alasaly@ksu.edu.sa

Mohamed A. Bencherif
mbencherif1@yahoo.com

Ahmed Alsanad
ahalsanad@ksu.edu.sa

[1] Department of Information Systems, College of Computer and Information Sciences, King Saud University, Riyadh 11543, Saudi Arabia

[2] Department of Computer Engineering, College of Computer and Information Sciences, King Saud University, Riyadh, Saudi Arabia

1 Introduction

Cloud computing is a new and promising method that combines a group of current technologies to manage businesses in various ways. The popularity of cloud computing is growing because it has the advantage of allowing customers flexibility and ease in deploying their applications, which makes the process of acquiring resources impressively simple. Additionally, users only pay on demand according to their actual resource usage (pay-per-use) [1–4]. Companies can use cloud computing for different purposes, ranging from executing batches of tasks to web application hosting [3]. Cloud computing is now one of the most common tools in many domains, particularly in

Published online: 11 November 2021

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

the domains of education, government research, and business, making it a significant tool for many users who depend on cloud-based applications in their personal lives and daily activities [5, 6].

In its simplest expression, cloud computing refers to the operation of storing, processing, and analyzing programs and data that are offered as a service to clients over the Internet [1, 7, 8]. Elasticity and dynamism are two central concepts in cloud computing. In this case, computing resources, such as CPUs, memory, and networks, are generally provided in the form of virtual machines (VMs).

As mentioned previously, elasticity is a major characteristic of cloud computing. Elasticity is related to a system's ability to adjust to fluctuations in workload and requests for resources [9]. In cloud computing, this elasticity property allows applications to gain and release resources dynamically. However, adapting to fluctuations in demand and allocating the correct amount of resources, such as in the case of CPU usage prediction, is not simple [3]. Noteworthily, it remains a challenge, and only a limited number of studies in the literature have addressed the problem.

A method for the autonomic provision of resources is used to exploit the elasticity property of the cloud, as well as to dynamically manage and optimally exploit computing resources. When changes occur in an application's workload, the available resources must be reconfigured to provide the necessary requirements and improve performance, all the while ensuring quality of service (QoS) requirements, avoiding service level agreements (SLAs) violations, and reducing the cost of renting cloud resources from software as a service (SaaS) providers [3, 10, 11].

In this work, we propose a hybrid resource provisioning method for cloud computing applications that is based on a combination of the autonomic computing concept and deep learning. Here, we focus on predicting the CPU usage of VMs in cloud data centers for future workload demands. IBM's reference model for autonomic computing, the control MAPE loop, was used in our implementation [12]. The autonomic model has four components: a monitor (M), an analyzer (A), a planner (P), and an executor (E). The monitor component is used to collect information metrics from sensors regarding resources and customers. The metrics gathered in the monitoring phase are processed by the analyzer component to predict the required number of requests for cloud services in the next time period. The planning component represents the framework's core component; it decides on the optimal actions that should be implemented (e.g., deciding how many VMs to add or remove for different cloud services). Finally, the executor component is responsible for executing actions, such as scaling in or out, which are determined by the planner component, in order to achieve effective resource provisioning and realize a trade-off between adhering to the SLA and minimizing costs [1, 13–16].

In our research, we propose an efficient deep learning model based on the diffusion convolutional recurrent neural network (DCRNN) [17]. The DCRNN model is based on the encoder-decoder neural network architecture. It implements its forecasting using sequence-to-sequence learning. The encoder-decoder architecture is built using a simple neural network variant known as gated recurrent units (GRUs). In GRUs, the diffusion convolution operation is used instead of matrix multiplication to create the DCRNN cell.

We apply DCRNN in the analysis phase to predict future demand, thus addressing workload fluctuations in the next interval. Our work makes two main contributions:

- We transformed CPU data from PlanetLab's workload into network graph form.
- We used a DCRNN-based workload forecasting approach to improve forecasting accuracy and minimize the error between the predicted and the actual workloads; we also included a forecast error computation that is the difference between the actual and predicted workloads.

This model can analyze the current workload and predict future demand, which depends on changes in workload, both autonomically and effectively without human intervention. Using this method, cloud services can be provisioned at a low cost with optimal CPU utilization.

Recently, deep learning approaches have outperformed other state-of-the-art techniques in many areas due to their hierarchical features and representations for learning [18]. At present, various deep learning methods have been developed, including stacked auto-encoders, deep belief networks (DBNs), and convolutional neural networks (CNNs). Unsupervised and supervised techniques have been used to train parameters for extracting multi-level features in deep learning [19]. Deep learning is part of machine learning (ML); it is an aspect of the modern generation of neural networks (NN), and it emerged as a concept from research on artificial neural networks (ANNs) [20].

Diverse deep learning methods have been developed in recent years to address workload changes [19, 21–23]. However, accurate forecasting is a complex task, especially in the presence of inconsistent and nonlinear workloads [23]. Moreover, cloud computing systems consist of many VMs, which increases the complexity of the prediction problem due to the correlations between the large workload data in these VMs. It has been difficult for prediction models to integrate accurate real-time forecasting because this further complicates system forecasting. Therefore, this paper uses a DCRNN workload forecasting approach to

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

improve forecasting accuracy, which cloud providers can exploit for service improvement. DCRNNs have been applied in previous studies, including traffic forecasting for road networks [16], public bus travel time estimation [24], and traffic load prediction on the links of a telecom network [25]. For all baseline metrics, DCRNNs have exhibited superior performance [16, 24, 25].

We have proposed to use the DCRNN as a new method to deal with both spatial and temporal behavior of distributed virtual machines (VMs). The DCRNN provide better predictions results, because it can handle long-term behavior in a better way as compared to other deep learning methods, due to the intrinsic nature of the recurrent networks that are part of the DCRNN. To the best of our knowledge—we are the first in this field to use DCRNN to predict future CPU usage for the purpose of correctly allocating allocation of resources for applications based on their demands.

The rest of this paper is organized as follows: in Sect. 2, related work is presented; Sect. 3 discusses the study's background; Sect. 4 details the research methodology; Sect. 5 presents and discusses the experiments and results; and finally, Sect. 6 offers concluding remarks.

2 Related work

Several methods have been developed to predict VM workloads in cloud computing, a review of which is given in [11, 26]. This paper concentrates on machine learning models for cloud workload forecasting. Therefore, this section reviews work related to forecasting cloud workloads based on machine learning approaches.

In [27], the authors proposed an adaptive prediction-based strategy model to measure and provide resources. In particular, they proposed a method combining linear regression (LR) and ANN to predict VM workloads. The authors trained and tested their prediction models using data from Amazon Elastic Compute Cloud's TPC-W. They demonstrated the efficiency of their prediction framework and claimed that it offered an accurate forecast of energy consumption.

In [28], the authors used an LR, support vector machine (SVM), and NN to forecast cloud resource provisioning. In their work, they predicted CPU utilization, response time, and throughput using data obtained from a VM-based web server and database servers. They demonstrated that SVM improved throughput and response time.

In [29], the authors presented a dynamic resource management algorithm that improved shared resource use. The algorithm's aim was to maximize resource utilization and minimize waste while ensuring SLA adherence. In [30], the authors proposed an approach for resource provisioning that was based on behavioral analysis, with the aim of analyzing the actions of cloud users and application behavior. The proposed method was based on two layers: a behavior layer and a translation layer. The translation layer computed the required transactions using the available resources and workload as its inputs. The application workload for the translation layer was predicted by the behavior layer.

In [31], the authors proposed an RNN for workload forecasting on cloud servers. The workload was the number of processes allocated to the servers. Their assessment results demonstrated that the RNN required less training data compared to a feedforward neural network. It was also able to anticipate rapid workload fluctuations better compared to the regression technique.

In [32], the authors introduced an improved fuzzy neural network (FNN) forecasting technique using an ensemble model and subtractive-fuzzy clustering-based fuzzy neural network (ESFCFNN) to predict requests for cloud resources. In their work, they employed various base predictor models for different periods. They demonstrated that the ESFCFNN model outperformed the FNN technique.

In [33], the authors introduced a novel VM workload prediction method by integrating an NN and a fuzzy logic system. In this approach, the NN was used to predict CPU usage patterns, whereas the fuzzy logic system was used to monitor future fluctuations in CPU workloads.

In [34], a reinforcement learning (RL) pattern was proposed to anticipate future application requests. In this technique, fuzzy logic models were used to increase the model's execution speed. RL's major advantage is that it does not require domain knowledge.

In [35], the authors presented a fuzzy auto-scaling method using workload forecasting for a massive multi-player online game (MMOG) application in a cloud computing environment, which relied on a support vector regression (SVR) algorithm. In addition, they proposed an algorithm that depended on a decision-maker for fuzzy auto-scaling to calculate the appropriate number of resources for assignment at every level of the MMOG application; specifically, this calculation used the forecasted workload and SLA for each user. Using this method, the authors efficiently reduced resource costs compared to the other approaches.

In [36], the authors introduced a multi-VM provisioning method. Their method combined the time-based billing aware multi-VM provisioning algorithm (TBAMP) approach and an autoregressive integrated moving average (ARIMA) model, where the ARIMADE enabled a reduction in cloud costs and the TBAMP technique estimated rental costs. This provisioning technique consisted of a partitioning module and a prediction module.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

In [37], the authors attempted to apply better workload prediction to minimize power costs. In their method, they analyzed and compared the expected performance of ARIMA, seasonal integrated ARMA (SARIMA), and fractionally integrated ARMA (ARFIMA) with another technique known as singular spectrum analysis, and they also used CPU, RAM, and network patterns gathered from the Wikimedia grid. The authors demonstrated that increasing the size of the input did not improve prediction ability, but calculation times for ARFIMA increased as the size of the input data increased.

In [38], the authors introduced a proactive strategy to manage dynamic forecasting of resource provisioning using an ARIMA model. Their method implemented a load analyzer component, which provided estimations for the other components that enabled a more accurate measure of resources. However, the ARIMA model had some limitations: it could not anticipate peak resource consumption.

In [39], the authors attempted to predict demand in future time intervals to prevent overloading. This is a sophisticated problem when historical data are not available for evaluation. In their work, the authors outlined a prediction technique using a genetic algorithm (GA) to determine a time series based on predictions. The workload predictions were made using the ARIMA and autoregressive moving average (ARMA) approaches. In addition, the authors used the Holt-Winters method to determine seasonality, but they did not present an improved cost model.

In [40], the authors proposed a tuned support vector regression (TSVR) scheme that cautiously selected three SVR parameters using hybrid GA and particle swarm optimization (PSO) methods. The scheme used a chaotic sequence to maximize predictive accuracy for memory usage and CPU allocation in a cloud environment. The method's predictive performance outperformed that of a conventional SVR method.

Many studies have been undertaken that have proposed estimation methods based on traditional machine learning approaches to generate predictions in adaptive and dynamic ways for the estimation of resource utilization in data centers in a cloud computing context [41–44].

In [45], Tofighy et al. proposed a framework containing a hybrid CPU load prediction model to accurately predict resource usage. Their framework used a Bayesian information standard to select the optimal constitutive resource utilization history. Additionally, it removed outliers using smooth filters and eliminated their negative impact.

Recently, researchers have been investigating deep learning techniques, and they have been found to outperform other state-of-the-art methods due to their hierarchical features [26]. Deep learning has achieved significant advances and successes in tackling numerous challenges in artificial intelligence. In particular, deep learning has outperformed other machine learning methods and generated promising results in diverse fields, including computer vision [46–48], transfer learning [49, 50], image classification and object recognition [46, 51–54], speech recognition [55–57], natural language understanding [58, 59], traffic flow prediction [60], industry [61], medical [62], and various other fields.

Deep learning approaches are suitable for forecasting long-term workloads, and it is possible to improve their performance by increasing the depth of the model and the size of the dataset used for training [63]. Various deep learning techniques have been developed to forecast workloads in data centers in the cloud computing context. For example, in [64], the authors attempted to find a relationship between the current and future workloads of VMs and achieved good precision. Moreover, they improved the performance of the training data and the operating functions, and they enhanced the number of layers. The authors used the expected information about workload to manage the VM and decision of migration is transmitted the plan is to the application provider that, in turn, will receive the customer request that is accepted and implement the appropriate strategy in order to assign the VM to the physical machines (PMs). To evaluate their deep learning model, the authors used PlanetLab data, and they demonstrated that a long short-term memory (LSTM) could improve workload prediction performance and outperform a CNN.

In [65], the authors trained bidirectional LSTM (BLSTM) models for resource prediction in data centers in a cloud computing context using a different set of parameters (that is, rather than only CPU allocation). In their work, they analyzed diverse resource metrics such as memory, disk space usage, and total page cache to forecast future CPU workload. The study considered both forward and backward dependencies. BLSTM models require a significant amount of training time as they need to learn double the number of parameters compared to a standard LSTM model.

In [21], the authors proposed a novel workload prediction approach based on deep learning for a VM. The deep learning prediction model consisted of a DBN and an RL model. Although the results indicated that the approach improved workload forecasting performance compared to other popular workload forecasting methods, the model was ineffective for predicting cloud workloads because it required the training of an enormous number of parameters.

In [22], using a cloud trace dataset obtained from Google, Zhang et al. proposed an approach based on a DBN to predict CPU and RAM utilization in data centers. In their proposed approach, they used backpropagation to reduce the loss function. In [19], a deep learning technique utilizing canonical polyadic decomposition (CPD) was

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

presented for workload forecasting in the cloud for industry informatics. The proposed method used deep learning techniques to acquire the most important features in the complicated workload data within the VMs, and CPD was applied to improve parameter choice and training effectiveness. However, forecasting accuracy was not sufficiently convincing.

Most of the above deep learning solutions are unsuitable for accurate forecasting in the presence of inconsistent and nonlinear workloads in cloud VMs, as well as correlations between the large workload data in these VMs. It has been difficult for any prediction model to integrate accurate real-time forecasting due to the fact that this further complicates system forecasting. In addition, other deep learning models did not consider both spatial and temporal behavior of distributed VMs. Therefore, this paper uses a DCRNN workload forecasting approach to improve forecasting accuracy and minimize the error between the predicted and the actual workloads.

3 Background

3.1 Autonomic framework

The proposed framework depends on IBM's autonomic model MAPE [12]. It implements an efficient resource provisioning framework model for cloud computing that consists of four components: a monitor, analyzer, planner, and executor, as shown in Fig. 1. A brief explanation of each component is given as follows:

- *Monitor* The monitor component collects information related to resources and users from sensors. After analysis, the effectors execute this collected information and the required changes.
- *Analyzer* The inputs to this component are the metrics gathered from the monitor component, including the number of VMs, the number of requests, and the VMs' CPU utilization. The output of the component is the predicted value for the cloud service that is presented to the SaaS provider.

In this study, the workload analyzer operates an estimating or forecasting procedure to anticipate future requests for cloud services over a specific time span. We propose a novel methodology for a VM workload analyzer based on the DCRNN technique to predict future requests and, hence, to manage the fluctuating workload.

- *Planner* The planner component is a major framework component. The prediction value is the input to this component, and the output is the selected action for the cloud service (e.g., scale-in or scale-out), which is offered by the SaaS provider.
- *Executor* This component implements the scaling decision determined by the planner component.

In our study, we focused on the analyzer component in our development of an efficient method for predicting the workload in the next time period based on the DCRNN model.

3.2 Diffusion convolutional recurrent neural network

DCRNN was used to model the dependency that formalizes the spatiotemporal CPU utilization learning problem [17].

3.2.1 CPU prediction problem

To solve the problem of forecasting CPU utilization, it is necessary to compare prior readings with future estimates of CPU usage made using a specific algorithm. This prediction is completed using more than 1000 VMs distributed across 700 different locations worldwide. The problem can be formulated as a weighted directed graph $= (, \varepsilon, W)$ where $\mathcal{V}$ is the number of nodes$|\mathcal{V}| = N$, ε is the number of edges, and $\boldsymbol{W} \epsilon \mathbb{R}^{N \times N}$ is a weighted adjacency matrix representing the nodes proximity. Denote the CPU flow observed on $\mathcal{G}$ as a graph signal $\boldsymbol{X} \epsilon \mathbb{R}^{N \times P}$, where P represents the number features of each node. Let $\boldsymbol{X}^{(t)}$ represent the graph signal observed at time t, the CPU forecasting problem aims to learn a function $h(\bullet)$ that maps T' historical graph signals to future T graph signals, given a graph $\mathcal{G}$[17]:

$$\left[\boldsymbol{X}^{(t-T'+1)}, \ldots, \boldsymbol{X}^{(t)}; \mathcal{G}\right] \xrightarrow{h(\bullet)} \left[\boldsymbol{X}^{(t+1)}, \ldots, \boldsymbol{X}^{(t+T)}\right] \quad (1)$$

4 Proposed methodology

We investigated several existing machine learning methods that can be used to predict the time series evolution of resource demands in cloud data centers. This is because our research problem is related to time series analysis. The deep learning technique applied in our research was a DCRNN with the ability to predict future workload over specific time intervals (in this case, 5, 15, 30, and 60 min).

The research methodology was divided into three phases: adaption of DCRNN for the CPU prediction problem, data generation, and training process. This section provides a detailed discussion of each phase. The study's objective was to develop an accurate hybrid deep learning model based on autonomic computing and a DCRNN model. The

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

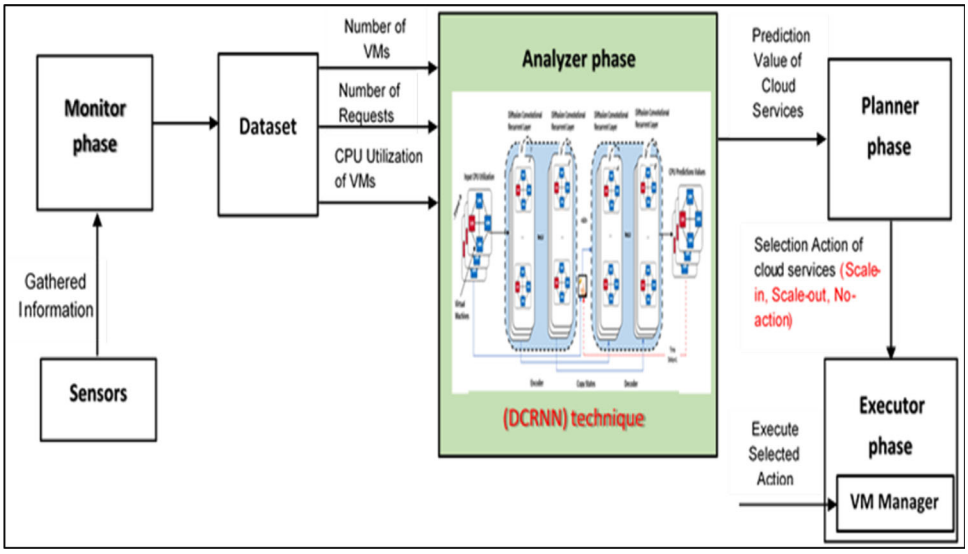


Fig. 1 Resource provisioning framework based on the control MAPE loop

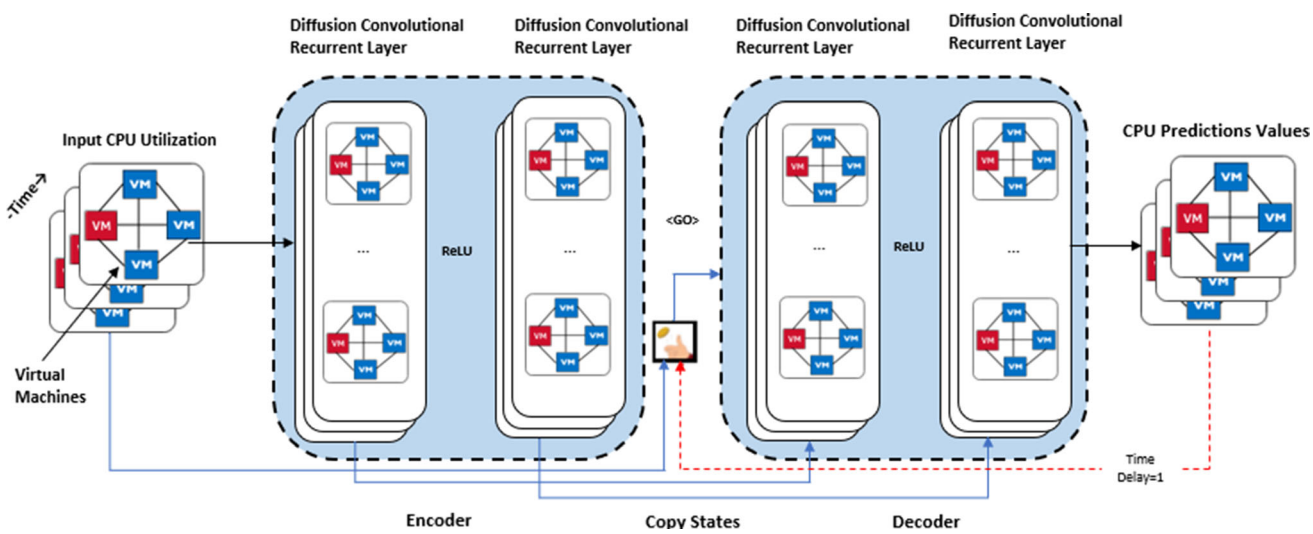


Fig. 2 DCRNN system architecture adapted from [17]

data for model input were collected from PlanetLab, while the output was the predicted value for the next time interval. In the next subsections, a description is given of the dataset used and the proposed model.

4.1 Adaption of DCRNN for CPU prediction problem

The Cloud Lab dataset consists of a set of distributed machines, where CPU utilization is monitored and recorded at successive periodic instants. Monitoring of these machines is a continuous process similar to a network traffic load. We developed a new approach to forecast CPU utilization as a network traffic monitoring problem, specifically using a DCRNN.

Each machine in PlanetLab is assimilated to a network sensor. Additionally, the values of the CPUs are associated with the network traffic, where each machine is an edge of a directed graph at a certain distance or location from the other machines, and CPU load is associated with network traffic weights.

The machines spread over the world have been positioned at a unit distance firstly and then positioned at a random location to mimic real situations. The diffusion graph is edge-based, and it uses an adjacency matrix to handle such distant sensors. In this way, CPU utilization is converted into a traffic forecasting problem.

The deep learning model uses two network models: an encoder and a decoder. Both networks contain a diffused convolution in a gated recurrent network. The goal of the

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Fig. 3 Data generation process

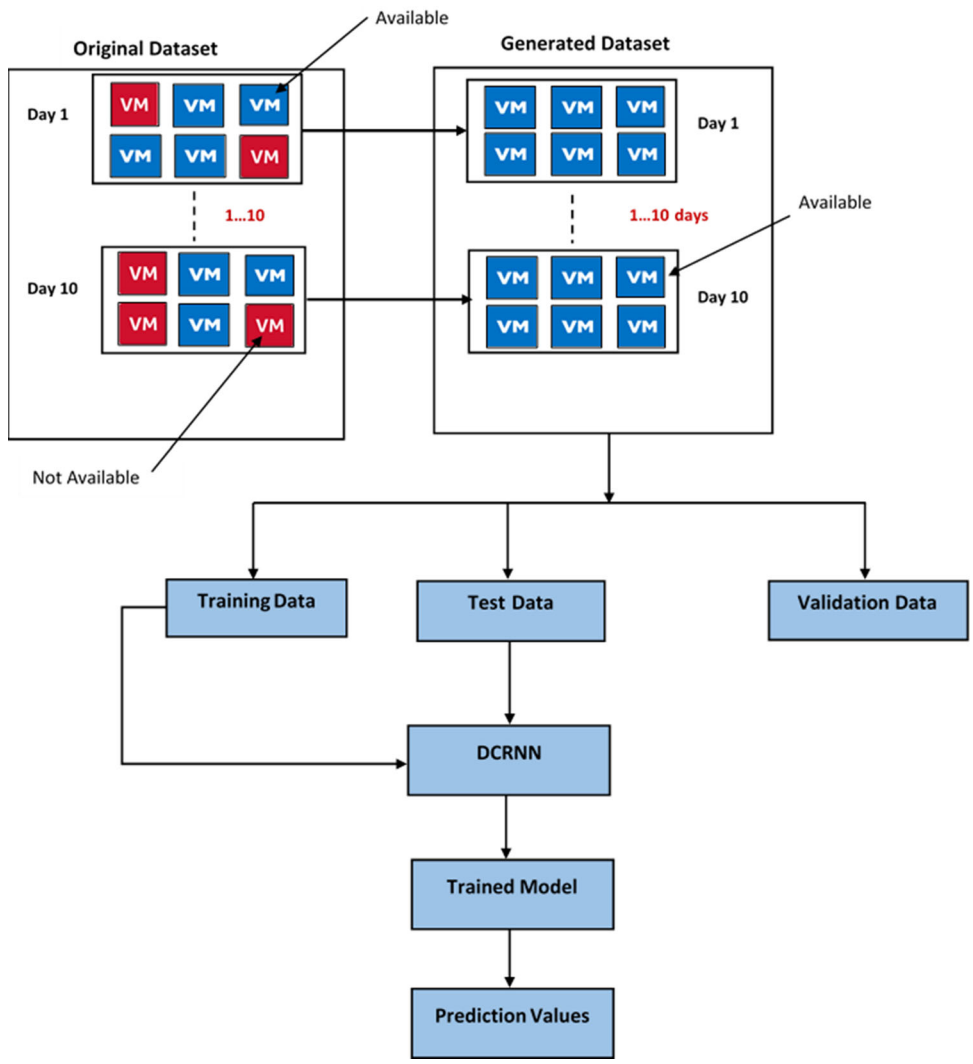


Table 1 Data generating methods description

Seq	Method	Description
1	Minus-one	Replace all the null value by -1 for all missing virtual machines
2	Small-values (0.01)	A small value replaces the zero values (0.01) for all missing virtual machines
3	Average-values	An Average value for all missing virtual machines replaces the null values
4	Ones-values	Replace all the null value by ones for all missing virtual machines

diffusion network is to predict CPU utilization values over the next 5, 15, 30, and 60 min. The network is a traffic prediction model that uses data about prior CPU utilization to predict future values. Experimental results indicate that such considerations lead to a good CPU utilization forecast model.

In this research, the proposed DCRNN architecture uses two layers, each with four diffusion convolutional gated recurrent units (DCGRU) (as shown in Fig. 2). The first layer is the encoder and the second is the decoder. Drawing on prior work on traffic prediction, we reformulated the workload prediction task for resource provisioning in a cloud computing context [17].

Our contribution converts CPU utilization data to satisfy the requirements of the DCRNN model, meaning that input data for CPU utilization were converted into a directed graph. From Fig. 2, we see that the PlanetLab dataset has many VMs that do not operate in all successive days

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 2 Characteristics for PlanetLab workload traces

Date	Number of VMs per day	
	Original data	Generating data
03/03/2011	1052	3596
06/03/2011	898	3596
09/03/2011	1061	3596
22/03/2011	1516	3596
25/03/2011	1078	3596
03/04/2011	1463	3596
09/04/2011	1358	3596
11/04/2011	1233	3596
12/04/2011	1054	3596
20/04/2011	1033	3596

(represented in red) and VMs that are operate on all days (represented in blue).

4.2 Dataset generation

The Cloud Lab dataset consists of a group of distributed machines for which CPU utilization is monitored and recorded at successive points in time. The monitoring of these machines is a continuous process similar to a network traffic load, which influenced our decision to approach the forecasting of CPU utilization as a network traffic monitoring problem using a DCRNN.

Each machine in the PlanetLab dataset was connected to a network graph, where each VM is an edge of a directed graph at a certain distance or location from the other VM, and the CPU load is associated with the network weights.

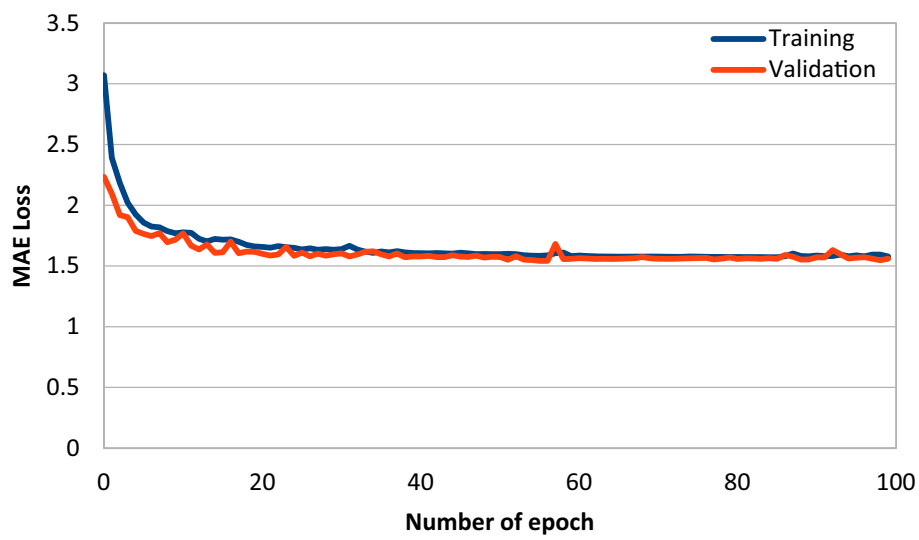


Fig. 4 DCRNN model training and validation loss using minus-one method

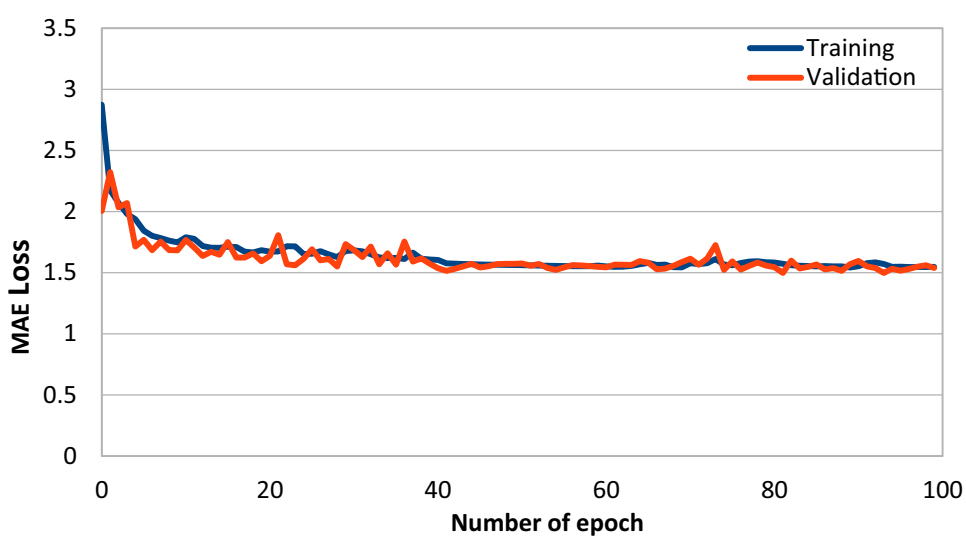


Fig. 5 DCRNN model training and validation loss using one-value method

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 3 Experimental formation

Num_rnn layers	2
Epochs	80
Batch size	16
Base learning rate	0.01
Optimizer	Adam
Num of Nodes	100
RNN_Units	4
Filter_type	dual_random_walk
Horizon	12
Dropout	0.2
Activation function	Relu

To generate the dataset for training and testing of the DCRNN, we used the CPU utilization from the PlanetLab dataset included in the cloud simulation tool CloudSim [66]. The dataset collected from PlanetLab included workloads for 10 different days with 288 samples per day. Figure 3 shows the generation of data for the training and testing process in our work.

This workload consisted of CPU utilization for more than 1,000 VMs collected every 5 min for 24 h over 10 days between March 3 to April 20, 2011.

The dataset contained 10 folders containing the data for each of the 10 days. Since there were many VMs that were not used on successive days, this led to many zeros in our deep learning model inputs, and such null inputs are of no benefit to the model. This is mainly because when large sequences of zeros appear, they do not actively contribute to the weight updates. The second problem comes from the computation of the MAPE at the end of training.

This metric requires actual truth values. To compensate for such null values, we opted for four different methods: using minus-one (−1), using small-values (0.01), using average-values and ones-values (1), and using a Z-score normalization process to rescale the features. Table 1 describes these four methods.

After generating the data, the new dataset was unified and there were 3596 VMs for all 10 days, whereas the original dataset contained a total of 11,746 VMs. The number of VMs differed for all 10 days ranging between 898 and 1516, as shown in Table 2.

4.3 Training process

In the proposed deep learning model, we conducted experiments to evaluate the performance of the DCRNN in terms of its efficiency in training and its accuracy in predicting the workload of the cloud-based VMs.

In this paper, we only used CPU utilization as the input data for our prediction model, and the output was the predicted value of CPU utilization at a future interval. We divided the data into three parts—training, testing, and validation—and allocated six days for training, two days for validation, and two days for testing (60%, 20%, and 20%, respectively, of the total available days of data).

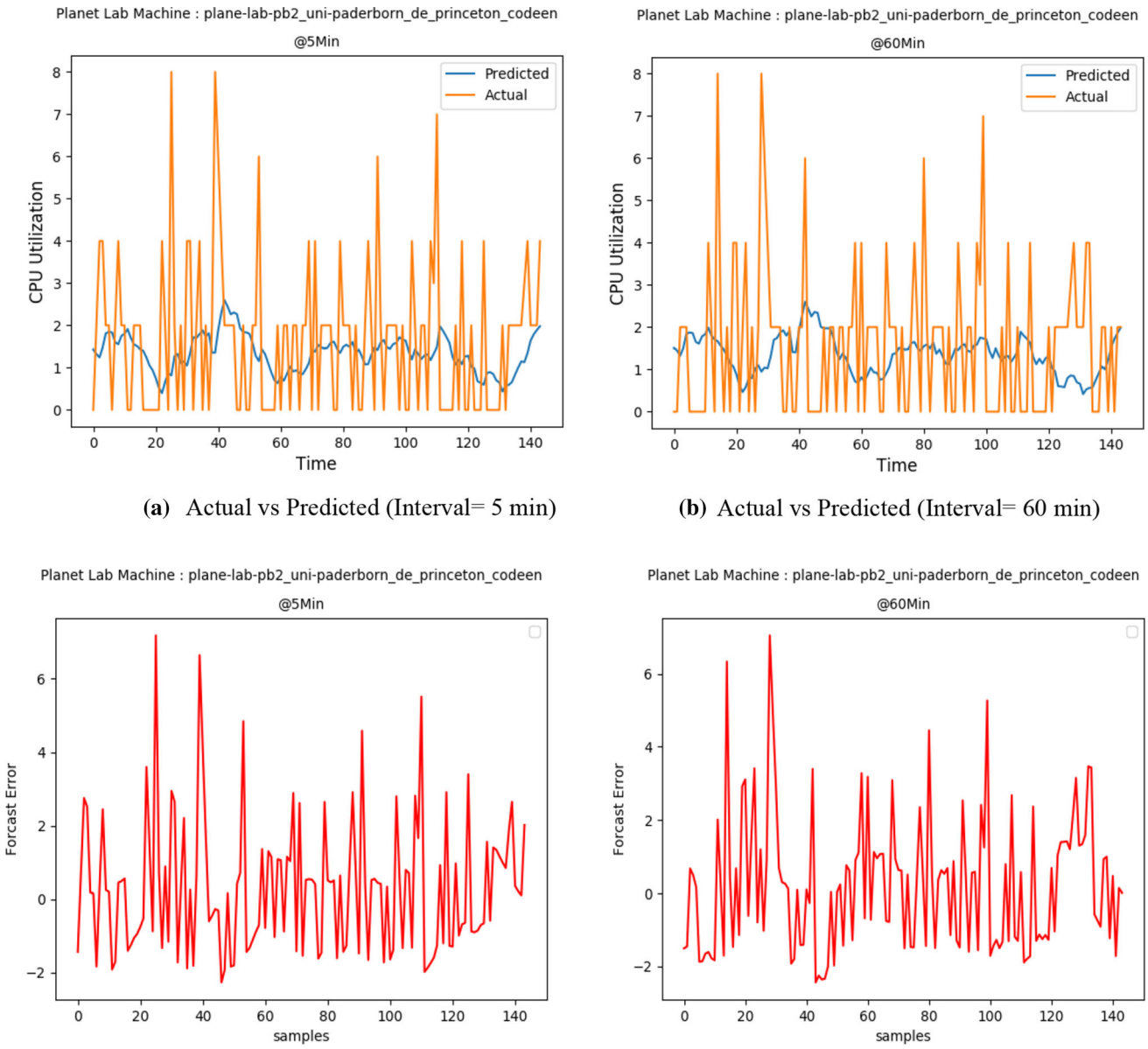
Before training and testing, we normalized the data to the [0,1] range. In the training phase, we conducted all experiments using four methods and two scaling methods, a Z-score (MS), and the rang scaling (P100) method at different time intervals (5, 15, 30, and 60 min).

We used the mean absolute error (MAE) loss function to estimate loss in the DCRNN model, as used in the original model. Figures 4 and 5 show the model's learning curve for our proposed minus-one-value and one-value methods. In the minus-one-value method, as shown in Fig. 4, we notice that the training loss was initially 3.070 and the validation loss was initially 2.234. After 100 epochs, these values decreased to 1.578 and 1.560 for the training and validation values, respectively. Similarly, in the one-value method, as shown in Fig. 5, we can notice that the training and validation values were initially 2.783 and 2.004 and decreased to 1.547 and 1.538, respectively. Through these

Table 4 MAPE/RMSE values for three models at different time intervals

Model		MAPE				RMSE			
		5 min	15 min	30 min	60 min	5 min	15 min	30 min	60 min
Neural Network [19]		0.26	0.27	0.31	0.33	10.06	11.85	12.14	12.95
DBN1 [21]		0.21	0.24	0.28	–	–	–	–	–
DBN2 [19]		0.22	0.24	0.26	0.32	10.26	11.18	10.37	11.65
CP-SAE [19]		0.21	0.22	0.25	0.26	9.17	9.64	10.29	9.97
Proposed Model (DCRNN)	Using 20 VMs	0.37	0.37	0.38	0.36	9.57	9.58	9.53	9.51
	Using all VMs	0.18	0.18	0.19	0.19	2.43	2.40	2.40	2.42

Content courtesy of Springer Nature, terms of use apply. Rights reserved.



(a) Actual vs Predicted (Interval= 5 min)

(b) Actual vs Predicted (Interval= 60 min)

(c) Forecast Residual (Interval= 5 min)

(d) Forecast Residual (Interval= 60 min)

Fig. 6 Prediction results for CPU Utilization when use 20 VMs

MAE loss values, we are assured that the model adequately learned from the CPU utilization data.

5 Experiments

When conducting our experiments, it was important to use real workload traces to evaluate the precision of the prediction algorithms. We used the CPU utilization of the PlanetLab dataset included in CloudSim [66], a cloud simulation tool.

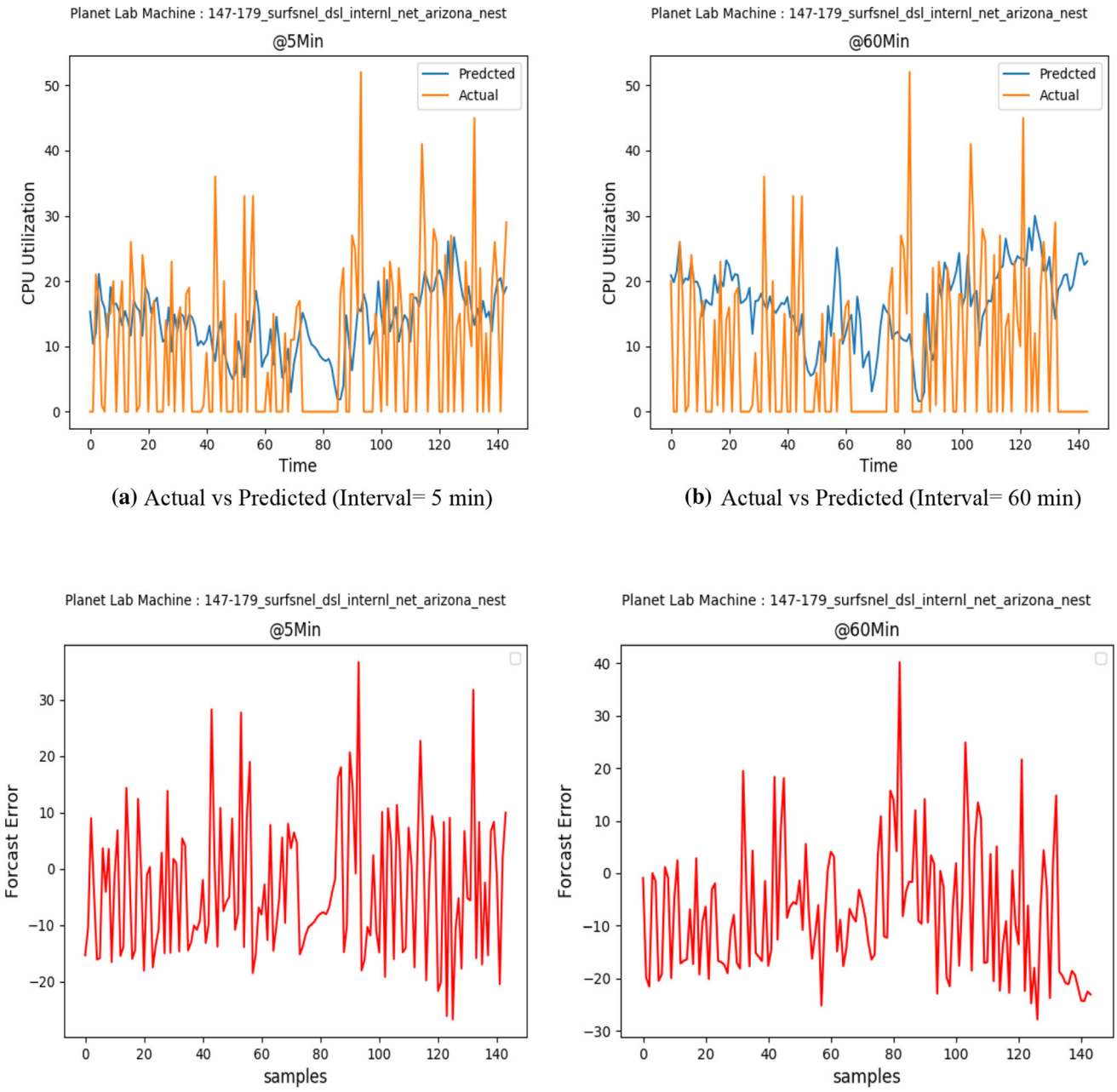
5.1 Experimental setup

The experiments were implemented on a server with a 3.10 GHz Xeon CPU, 16 GB memory, and 2 TB storage. Python was used to implement the algorithms. Table 3 shows the parameters used in our work.

5.2 Performance evaluation

To evaluate our experiments, we used two metrics: the mean absolute percentage error (MAPE) and the root-mean-square error (RMSE). These are well-known metrics

Content courtesy of Springer Nature, terms of use apply. Rights reserved.



(a) Actual vs Predicted (Interval= 5 min)

(b) Actual vs Predicted (Interval= 60 min)

(c) Forecast Residual (Interval= 5 min)

(d) Forecast Residual (Interval= 60 min)

Fig. 7 Prediction results for CPU Utilization when use all VMS

used to measure the accuracy of prediction models, and they are represented by

$$MAPE = \frac{1}{n}\sum_{i=1}^{n}\left|\frac{A_t - F_t}{A_t}\right| \tag{2}$$

$$RMSE = \sqrt{\frac{\sum_{i=1}^{n}(A_t - F_t)^2}{n}} \tag{3}$$

where A_t is the actual workload value, F_t is the predictive workload of the ith VM, and n is the number of VMs. In turn, we trained the deep learning algorithm with different parameters to optimize performance.

5.3 Results and discussion

We compared workload prediction accuracy with other deep learning methods from prior works [19, 21]. In Zhang et al. [19] introduced a method for estimating VM CPU utilization based on a deep belief network (DBN2), a polyadic canonical decomposition autoencoder model (CP-

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

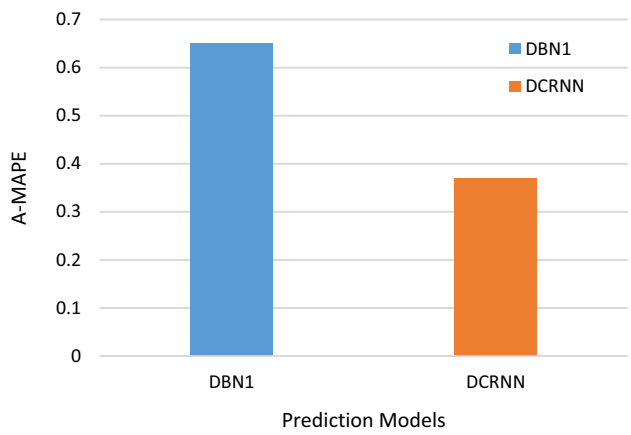


Fig. 8 Comparison between DCRNN and DBN1 using A-MAPE for 20 VMs

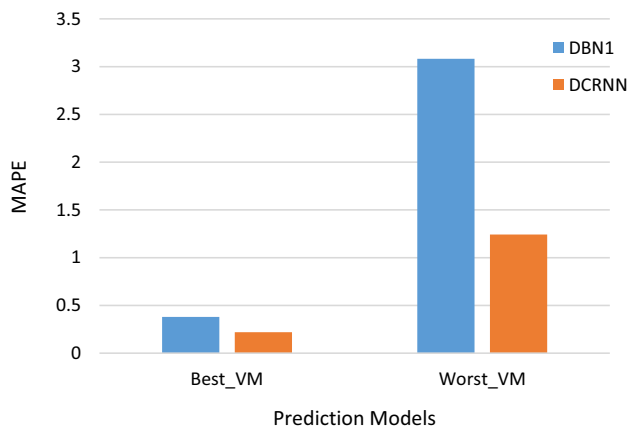


Fig. 9 Comparison between DCRNN and DBN1 using best and worst VM

SAE), and a traditional ANN. A comparison of the prediction accuracy measures, MAPE and RMSE, between our proposed method and three different models (specifically, CP-SAE, DBN, and the traditional NN) is shown in Table 4.

We implemented all our methods on 20 VMs that ran continuously for 10 days and on all VMs in PlanetLab.

The results indicate that the proposed method achieved superior results compared to the other methods, especially when all VMs were used; it was associated with the lowest values for MAPE and RMSE. For example, the MAPE value achieved by the proposed method was 0.18 for the 15-min time interval, which was less than the values of the other methods; the corresponding MAPE values yielded by the CP-SAE, DBN, and traditional neural network models were 0.22, 0.24, and 0.27, respectively. We can observe from the results that the proposed method yielded better solutions in terms of accuracy on multiple VMs. In addition, the proposed method exhibited favorable predictive

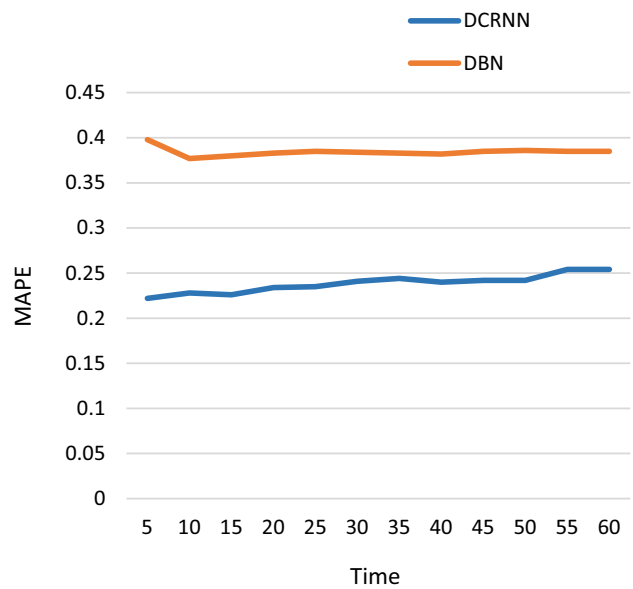


Fig. 10 Comparison between DCRNN and DBN1 using best VM for every five minutes during one hour

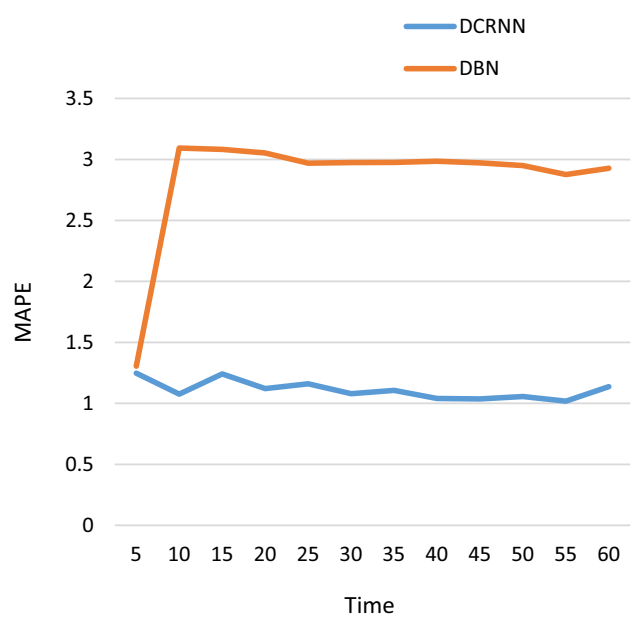


Fig. 11 Comparison between DCRNN and DBN using worst VM for every five minutes during one hour

accuracy for long workload predictions (e.g., the time interval of 60 min), which means that the proposed method can use salient features to learn effectively.

Figure 6a and b shows the prediction results of CPU usage for the actual and predicted workloads for one of the VM in the future one testing day at the 5- and 60-min intervals. As seen from Fig. 6, which illustrates the

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 5 Statistical significance results of MAPE between DCRNN and all other methods

Time Interval	MAPE			
	NN	DBN1	DBN2	CP-SAE
5 s	< 0.00001	1.64E-01	1.52E-01	1.63E-01
15 s	< 0.00001	< 0.00001	< 0.00001	1.53E-01
30 s	< 0.00001	< 0.00001	< 0.00001	< 0.00001
60 s	< 0.00001	< 0.00001	–	< 0.00001

Table 6 Statistical significance results of RMSE between DCRNN and all other methods

Time Interval	RMSE			
	NN	DBN1	DBN2	CP-SAE
5 min	< 0.00001	–	< 0.00001	< 0.00001
15 min	< 0.00001	–	< 0.00001	< 0 .00001
30 min	< 0.00001	–	< 0.00001	< 0 .00001
60 min	< 0.00001	–	< 0.00001	< 0.00001

workload changes at the various times in one day, it can be readily confirmed that the predictor learned the trends of the true mean CPU usage over time. The predictions closely approximate the actual CPU usage in most situations, and the minimum number of residuals are produced. The forecast errors for the 5- and 60-min prediction intervals are illustrated in Fig. 6c and d, respectively. This demonstrates that the proposed method performed competitively, where we observe that the predicted CPU usage closely approximated the actual CPU usage. Additionally, variation in usage was accurately predicted, which confirms that the proposed method was the most effective.

In addition, Fig. 7a and b shows CPU usage performance for one of the VMs when the proposed method was applied to all VMs. The predictions are close to the actual CPU usage values in most cases, and, as we can notice from Fig. 7c and d, the forecast error is higher when using all VMs compared to when using 20 VMs for both the 5- and 60-min intervals.

To show robustness of our proposed prediction model, we examined a detailed case study with the state-of-the-art technique- DBN1 method [21]. We used average MAPE (A-MAPE) for 20 VMs as the performance metric. The experimental result is shown in Fig. 8. We can see that the proposed DCRNN model outperforms the DBN1 method.

In addition, we studied best case and worst case of VMs to compare with DBN1 method in specific time interval (15 min) on our data set. We used a set of parameters with epochs 80, 16 batch size, 0.01 learning rate and MS-0ne scaling. The experimental result is depicted in Fig. 9. As we can see from Fig. 9, our proposed method (DCRNN) performs well as compared to the in DBN1 algorithm [21].

Besides, Figs. 10 and 11 show a comparison between the best and worst case of single VM scenario on proposed model (DCRNN) with the best and worst case scenario of single VM in DBN1 model for every five minutes during one hour. We notice that our method still outperforms the DBN1 approach in both cases. From these above case studies, it is clear that the proposed DCRNN model is effective and achieves competitive results in terms of robustness.

The ANOVA test was also used to test the significance of the improvement in the results on all VMs. Tables 5 and 6 present the significance metrics with a 0.05 significance level for all VMs on the PlanetLab dataset for the DCRNN method and the other models. In Table 5, the results indicate that the results for the proposed method were statistically significant in terms of the MAPE metric for all competing methods in most cases, but in some cases, the

Table 7 Comparison of the effect of different parameters on the results for 20 VMs

Ones-values method/15 min	Batch Size = 32 Learning Rate = 0.01				Batch Size = 32 Learning Rate = 0.001			
	Scaling Type				Scaling Type			
	MS		P100		MS		P100	
	RMSE	MAPE	RMSE	MAPE	RMSE	MAPE	RMSE	MAPE
Epochs (20)	9.43	0.39	9.46	0.41	9.60	0.38	9.59	0.41
Epochs (40)	9.48	0.38	9.45	0.40	9.45	0.39	9.40	0.40
Epochs (60)	9.47	0.38	9.33	0.40	9.46	0.40	9.40	0.41
Epochs (80)	9.51	0.38	9.39	0.40	9.51	0.40	9.35	0.41
Epochs (100)	9.50	0.38	9.34	0.41	9.37	0.41	9.35	0.41

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 8 Comparison of the effect of different parameters on the results for all VMs

Ones-values method/15 min	Batch Size = 32 Learning Rate = 0.01				Batch Size = 32 Learning Rate = 0.001			
	Scaling Type				Scaling Type			
	MS		P100		MS		P100	
	RMSE	MAPE	RMSE	MAPE	RMSE	MAPE	RMSE	MAPE
Epochs (20)	2.42	0.20	2.43	0.22	2.61	0.19	2.52	0.24
Epochs (40)	2.43	0.20	2.43	0.22	2.58	0.24	2.48	0.27
Epochs (60)	2.46	0.24	2.43	0.24	2.50	0.19	2.42	0.21
Epochs (80)	2.42	0.19	2.41	0.20	2.46	0.21	2.43	0.24
Epochs (100)	2.43	0.19	2.40	0.20	2.46	0.22	2.40	0.21

Table 9 Comparison of the effect of different parameters on the results for 20 VMs

Ones-values method/15 min	Batch = 16 Learning rate = 0.01				Batch Size = 16 Learning rate = 0.001			
	Scaling Type				Scaling Type			
	MS		P100		MS		P100	
	RMSE	MAPE	RMSE	MAPE	RMSE	MAPE	RMSE	MAPE
Epochs (20)	9.456	0.39	9.463	0.391	9.387	0.413	9.343	0.407
Epochs (40)	9.564	0.372	9.333	0.398	9.467	0.400	9.376	0.409
Epochs (60)	9.282	0.449	9.250	0.445	9.420	0.398	9.401	0.394
Epochs (80)	9.250	0.459	9.262	0.446	9.495	0.368	9.371	0.404
Epochs (100)	9.288	0.426	9.269	0.442	9.369	0.392	9.392	0.394

Table 10 Comparison of the effect of different parameters on the results for all VMs

Ones-values method/15 min	Batch = 16 Learning rate = 0.01				Batch Size = 16 Learning rate = 0.001			
	Scaling Type				Scaling Type			
	MS		P100		MS		P100	
	RMSE	MAPE	RMSE	MAPE	RMSE	MAPE	RMSE	MAPE
Epochs (20)	2.445	0.197	2.451	0.232	2.515	0.206	2.459	0.261
Epochs (40)	2.454	0.200	2.430	0.222	2.546	0.223	2.421	0.220
Epochs (60)	2.430	0.190	2.401	0.194	2.423	0.195	2.427	0.210
Epochs (80)	2.449	0.183	2.399	0.206	2.437	0.205	2.478	0.266
Epochs (100)	2.438	0.184	2.412	0.200	2.458	0.212	2.409	0.202

proposed method's results were not significant compared to DBN1, DBN2, and CP-SAE in the intervals of 5 and 15 min (CP-SAE).

Table 6 indicates that the proposed method's results were statistically significant in terms of the RMSE metric for all competing methods in all cases.

Additionally, we conducted more experiments using different parameters to obtain optimal results using 20 VMs and all VMs. Tables 7, 8, 9, 10, 11, 12 and 13 provide details of the results.

The following three parameters were used to compare the experiments: batch size, epochs, and learning rate. Initially, we set the batch size as 32 and established two

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 11 Comparison of the effect of different parameters on the results for 20 VMs

Batch = 16 Learning rate = 0.01	MAPE				RMSE			
	5 min	15 min	30 min	60 min	5 min	15 min	30 min	60 min
Epochs (20)	0.392	0.392	0.382	0.382	9.435	9.456	9.429	9.377
Epochs (40)	0.385	0.387	0.385	0.376	9.461	9.465	9.429	9.367
Epochs (60)	0.384	0.378	0.372	0.376	9.546	9.607	9.583	9.492
Epochs (80)	**0.372**	**0.373**	**0.376**	**0.363**	**9.566**	**9.579**	**9.534**	**9.507**
Epochs (100)	0.373	0.371	0.367	0.370	9.480	9.514	9.521	9.486

Bold values represent the best obtained results

Table 12 Comparison of the effect of different parameters on the results for all VMs

Batch = 16 Learning rate = 0.01	MAPE				RMSE			
	5 min	15 min	30 min	60 min	5 min	15 min	30 min	60 min
Epochs (20)	0.210	0.197	0.204	0.204	2.450	2.455	2.463	2.475
Epochs (40)	0.212	0.206	0.200	0.196	2.429	2.439	2.437	2.433
Epochs (60)	0.196	0.190	0.189	0.192	2.420	2.430	2.235	2.438
Epochs (80)	**0.183**	**0.183**	**0.185**	**0 .186**	**2.425**	**2.399**	**2.400**	**2.416**
Epochs (100)	0.184	0.184	0.185	0.189	2.423	2.438	2.446	2.452

Bold values represent the best obtained results

Table 13 Comparison of the effect of different parameters on the results for 20 VMs/all VMs

Ones-values method/15 min	All VMS Batch Size = 64 Learning Rate = 0.01				20 VMS Batch Size = 64 Learning Rate = 0.01			
	Scaling Type				Scaling Type			
	MS		P100		MS		P100	
	RMSE	MAPE	RMSE	MAPE	RMSE	MAPE	RMSE	MAPE
Epochs (20)	2.629	0.370	2.543	0.337	9.468	0.403	9.339	0.403
Epochs (40)	2.555	0.204	2.504	0.267	9.424	0.404	9.457	0.402
Epochs (60)	2.442	0.206	2.510	0.202	9.476	0.400	9.358	0.410
Epochs (80)	2.443	0.194	2.402	0.198	9.583	0.381	9.385	0.397
Epochs (100)	2.434	0.191	2.431	0.235	9.568	0.381	9.361	0.404

Table 14 Summary of the best obtained results from all previous experiment

Batch = 16 Learning rate = 0.01		MAPE				RMSE			
		5 min	15 min	30 min	60 min	5 min	15 min	30 min	60 min
Epochs (80)	20 VMs	0.372	0.373	0.376	0.363	9.566	9.579	9.534	9.507
	All VMs	0.183	0.183	0.185	0.186	2.425	2.399	2.400	2.416

learning rates of 0.01 and 0.001. Further to the comparative analysis, we set a batch size of 16 with two learning rates of 0.01 and 0.001. Finally, we set a batch size of 64 with learning rate of 0.01. The results are shown in following tables:

We have noticed that through the experiments and comparisons above the best results were at epochs 80 with

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

16 batch size, 0.01 learning rate, and MS scaling method as shown in Tables 11 and 12. The best results obtained on 20 VMs and all VMs are summarized in Table 14.

6 Conclusion and future work

This paper presented a deep learning-based workload prediction model for resource provisioning in an autonomic cloud computing environment. We focused on forecasting CPU usage for virtual machines in cloud data centers to meet future workload demands. An efficient deep learning model based on diffusion convolutional recurrent neural networks (DCRNN) was proposed to predict future demands for CPU usage and address workload fluctuations for future time intervals. The main novelty of the proposed approach is its ability to capture important topological properties of the network that significantly and positively influence forecasting accuracy. The model enables both an analysis of the current workload and the prediction of future demand. To validate the effectiveness of the proposed DCRNN model, various experiments were conducted using the PlanetLab dataset. The experimental results clearly demonstrated that the proposed approach outperformed other existing deep learning approaches. For example, a reduction of the MAPE from 0.32 to 0.19 and RMSE from 11.65 to 2.42, as compared to DBN2 (60 Min), was observed. These promising results suggest that the proposed DCRNN-based forecasting model was useful for resource provisioning in the autonomic cloud computing platform. However, in the case of very fluctuant workloads with random distribution, the performance of our proposed model might vary. In the future, we plan to use other datasets with randomly fluctuating workload traces to evaluate the forecasting or prediction effectiveness of our proposed model.

Acknowledgements This work was supported by King Saud University, Riyadh, Saudi Arabia, under Researchers Supporting Project number RSP-2021/18.

Declarations

Conflict of interest There is no conflict of interests among the authors.

References

1. Al-Asaly MS, Hassan MM, Alsanad A (2019) A cognitive/intelligent resource provisioning for cloud computing services: opportunities and challenges. Soft Comput 23:9069–9081
2. Zhang Q, Cheng L, Boutaba R (2010) Cloud computing: state-of-the-art and research challenges. J Internet Serv Appl 1:7–18
3. Lorido-Botran T, Miguel-Alonso J, Lozano JA (2014) A review of auto-scaling techniques for elastic applications in cloud environments. J Grid Comput 12:559–592
4. Buyya R, Yeo CS, Venugopal S, Broberg J, Brandic I (2009) Cloud computing and emerging IT platforms: vision, hype, and reality for delivering computing as the 5th utility. Futur Gener Comput Syst 25:599–616
5. Buyya R, Vecchiola C, Selvi ST (2013) Mastering cloud computing: foundations and applications programming. Newnes
6. Chandrasekaran K (2014) Essentials of cloud computing. CrC Press, Boca Raton
7. Mustafa S, Nazir B, Hayat A, Khan AUR, Madani SA (2015) Resource management in cloud computing: taxonomy, prospects, and challenges. Comput Electr Eng 47:186–203
8. Manvi SS, Krishna Shyam G (2014) Resource management for Infrastructure as a Service (IaaS) in cloud computing: a survey. J Netw Comput Appl 41:424–440
9. Herbst NR, Kounev S, Reussner R (2013) Elasticity in cloud computing: What it is, and what it is not. In: 10th Int Conf Auton Comput (ICAC 13). pp 23–27
10. Qavami HR, Jamali S, Akbari MK, Javadi B (2014) Dynamic resource provisioning in cloud computing: a heuristic markovian approach. Lect Notes Inst Comput Sci Soc Telecommun Eng LNICST 133:102–111
11. Amiri M, Mohammad-Khanli L (2017) Survey on prediction models of applications for resources provisioning in cloud. J Netw Comput Appl 82:93–113
12. Kephart JO, Chess DM (2003) The vision of autonomic computing. Computer (Long Beach Calif) 36:41–50
13. Ghobaei-Arani M, Jabbehdari S, Pourmina MA (2018) An autonomic resource provisioning approach for service-based cloud applications: a hybrid approach. Futur Gener Comput Syst 78:191–210
14. Jacob B, Lanyon-Hogg R, Nadgir DK, Yassin AF (2004) A practical guide to the IBM autonomic computing toolkit. IBM Redbooks 4
15. Maurer M, Brandic I, Sakellariou R (2013) Adaptive resource configuration for Cloud infrastructure management. Futur Gener Comput Syst 29:472–487
16. Mateen M, Hayat S, Tehreem T, Akbar MA (2020) A self-adaptive resource provisioning approach using fuzzy logic for cloud-based applications. Int J Comput Digit Syst 9
17. Li Y, Yu R, Shahabi C, Liu Y (2017) Diffusion convolutional recurrent neural network: data-driven traffic forecasting. arXiv 1–16
18. Zhang Q, Yang LT, Chen Z, Li P (2018) A survey on deep learning for big data. Inf Fusion 42:146–157
19. Zhang Q, Yang LT, Yan Z, Chen Z, Li P (2018) An efficient deep learning model to predict cloud workload for industry informatics. IEEE Trans Ind Inform 14:3170–3178
20. Deng L, Yu D (2014) Deep learning: methods and applications. Found Trends Signal Process 7:197–387
21. Qiu F, Zhang B, Guo J (2016) A deep learning approach for VM workload prediction in the cloud. In: 2016 IEEE/ACIS 17th Int Conf Softw Eng Artif Intell Netw Parallel/Distributed Comput SNPD 2016 319–324
22. Zhang W, Duan P, Yang LT, Xia F, Li Z, Lu Q, Gong W, Yang S (2017) Resource requests prediction in the cloud computing environment with a deep belief network. Softw Pract Exp 47:473–488
23. Kumar J, Singh AK, Buyya R (2021) Self directed learning based workload forecasting model for cloud resource management. Inf Sci (Ny) 543:345–366
24. Tran L, Mun MY, Lim M, Yamato J, Huh N, Shahabi C (2020) DeepTRANS: a deep learning system for public bus travel time

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

estimation using traffic forecasting. Proc VLDB Endow 13:2957–2960
25. Andreoletti D, Troia S, Musumeci F, Giordano S, Maier G, Tornatore M (2019) Network traffic prediction based on diffusion convolutional recurrent neural networks. INFOCOM 2019 - IEEE Conf Comput Commun Work INFOCOM WKSHPS 2019 246–251
26. Masdari M, Khoshnevis A (2020) A survey and classification of the workload forecasting methods in cloud computing. Cluster Comput 23:2399–2424
27. Islam S, Keung J, Lee K, Liu A (2012) Empirical prediction models for adaptive resource provisioning in the cloud. Futur Gener Comput Syst 28:155–162
28. Bankole AA, Ajila SA (2013) Predicting cloud resource provisioning using machine learning techniques. Can Conf Electr Comput Eng 31–34
29. Garg SK, Toosi AN, Gopalaiyengar SK, Buyya R (2014) SLA-based virtual machine management for heterogeneous workloads in a cloud datacenter. J Netw Comput Appl 45:108–120
30. Kousiouris G, Menychtas A, Kyriazis D, Gogouvitis S, Varvarigou T (2014) Dynamic, behavioral-based estimation of resource provisioning based on high-level application terms in Cloud platforms. Futur Gener Comput Syst 32:27–40
31. Chang YC, Chang RS, Chuang FW (2014) A predictive method for workload forecasting in the cloud environment. Lect Notes Electr Eng. https://doi.org/10.1007/978-94-007-7262-5_65
32. Chen Z, Zhu Y, Di Y, Feng S (2015) Self-adaptive prediction of cloud resource demands using ensemble model and subtractive-fuzzy clustering based fuzzy neural network. Comput Intell Neurosci. https://doi.org/10.1155/2015/919805
33. Ramezani F, Naderpour M (2017) A fuzzy virtual machine workload prediction method for cloud environments. IEEE Int Conf Fuzzy Syst. https://doi.org/10.1109/FUZZ-IEEE.2017.8015450
34. Amiri M, Feizi-Derakhshi MR, Mohammad-Khanli L (2017) IDS fitted Q improvement using fuzzy approach for resource provisioning in cloud. J Intell Fuzzy Syst 32:229–240
35. Khorsand R, Ghobaei-Arani M, Ramezanpour M (2018) WITHDRAWN: A fuzzy auto-scaling approach using workload prediction for MMOG application in a cloud environment. Simul Model Pract Theory 1
36. Li S, Wang Y, Qiu X, Wang D, Wang L (2013) A workload prediction-based multi-vm provisioning mechanism in cloud computing. In: 2013 15th Asia-Pacific Netw. Oper. Manag. Symp. IEEE, pp 1–6
37. Kumar AS, Mazumdar S (2016) Forecasting HPC workload using ARMA models and SSA. In: 2016 Int. Conf. Inf. Technol. IEEE, pp 294–297
38. Calheiros RN, Masoumi E, Ranjan R, Buyya R (2015) Workload prediction using ARIMA model and its impact on cloud applications' QoS. IEEE Trans Cloud Comput 3:449–458
39. Messias VR, Estrella JC, Ehlers R, Santana MJ, Santana RC, Reiff-Marganiec S (2016) Combining time series prediction models using genetic algorithm to autoscaling Web applications hosted in the cloud infrastructure. Neural Comput Appl 27:2383–2406
40. Barati M, Sharifian S (2015) A hybrid heuristic-based tuned support vector regression model for cloud load prediction. J Supercomput 71:4235–4259
41. Baig SUR, Iqbal W, Berral JL, Erradi A, Carrera D (2019) Adaptive prediction models for data center resources utilization estimation. IEEE Trans Netw Serv Manag 16:1681–1693
42. Nikravesh AY, Ajila SA, Lung CH (2015) Towards an autonomic auto-scaling prediction system for cloud resource provisioning. In: Proc - 10th Int Symp Softw Eng Adapt Self-Managing Syst SEAMS 2015 35–45
43. Ran Y, Yang J, Zhang S, Xi H (2017) Dynamic IaaS computing resource provisioning strategy with QoS constraint. IEEE Trans Serv Comput 10:190–202
44. Moreno-Vozmediano R, Montero RS, Huedo E, Llorente IM (2019) Efficient resource provisioning for elastic Cloud services based on machine learning techniques. J Cloud Comput. https://doi.org/10.1186/s13677-019-0128-9
45. Tofighy S, Rahmanian AA, Ghobaei-Arani M (2018) An ensemble CPU load prediction algorithm using a Bayesian information criterion and smooth filters in a cloud computing environment. Softw Pract Exp 48:2257–2277
46. Krizhevsky A, Sutskever I, Hinton GE (2017) Imagenet classification with deep convolutional neural networks. Commun ACM 60:84–90
47. Ciregan D, Meier U, Schmidhuber J (2012) Multi-column deep neural networks for image classification. Proc IEEE Comput Soc Conf Comput Vis Pattern Recognit 3642–3649
48. Sermanet P, Eigen D, Zhang X, Mathieu M, Fergus R, LeCun Y (2014) Overfeat: integrated recognition, localization and detection using convolutional networks. 2nd Int. Conf. Learn. Represent. ICLR 2014 - Conf. Track Proc
49. Ciresan DC, Meier U, Schmidhuber J (2012) Transfer learning for Latin and Chinese characters with deep neural networks. Proc Int Jt Conf Neural Networks. https://doi.org/10.1109/IJCNN.2012.6252544
50. Ren JS, Xu L (2015) On vectorization of deep convolutional neural networks for vision tasks. Proc Natl Conf Artif Intell 3:1840–1846
51. Farabet C, Couprie C, Najman L, Lecun Y (2013) Learning hierarchical features for scene labeling. IEEE Trans Pattern Anal Mach Intell 35:1915–1929
52. Tompson J, Jain A, LeCun Y, Bregler C (2014) Joint training of a convolutional network and a graphical model for human pose estimation. Adv Neural Inf Process Syst 2:1799–1807
53. Szegedy C, Liu W, Jia Y, Sermanet P, Reed S, Anguelov D, Erhan D, Vanhoucke V, Rabinovich A (2015) Going deeper with convolutions. In: Proc IEEE Conf Comput Vis pattern Recognit pp 1–9
54. Hayat M, Bennamoun M, An S (2015) Deep reconstruction models for image set classification. IEEE Trans Pattern Anal Mach Intell 37:713–727
55. Mikolov T, Deoras A, Povey D, Burget L, Černocký J (2011) Strategies for training large scale neural network language models. In: 2011 IEEE Work Autom Speech Recognit Understanding, ASRU 2011, Proc 196–201
56. Hinton G, Deng L, Yu D, Dahl GE, Mohamed A, Jaitly N, Senior A, Vanhoucke V, Nguyen P, Sainath TN (2012) Deep neural networks for acoustic modeling in speech recognition: the shared views of four research groups. IEEE Signal Process Mag 29:82–97
57. Sainath TN, Mohamed AR, Kingsbury B, Ramabhadran B (2013) Deep convolutional neural networks for LVCSR. ICASSP, IEEE Int Conf Acoust Speech Signal Process - Proc 8614–8618
58. Collobert R, Weston J, Bottou L, Karlen M, Kavukcuoglu K, Kuksa P (2011) Natural language processing (almost) from scratch. J Mach Learn Res 12:2493–2537
59. Mikolov T, Sutskever I, Chen K, Corrado GS, Dean J (2013) Distributed representations of words and phrases and their compositionality. Adv Neural Inf Process Syst 26:3111–3119
60. Lv Y, Duan Y, Kang W, Li Z, Wang FY (2015) Traffic flow prediction with big data: a deep learning approach. IEEE Trans Intell Transp Syst 16:865–873
61. Dey S, Pratiher S, Mukherjee CK, Banerjee S (2020) Solarisnet: a deep regression network for solar radiation prediction. Mausam 71:443–450

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

62. Cole JH, Poudel RPK, Tsagkrasoulis D, Caan MWA, Steves C, Spector TD, Montana G (2017) Predicting brain age with deep learning from raw imaging data results in a reliable and heritable biomarker. Neuroimage 163:115–124
63. Lecun Y, Bengio Y, Hinton G (2015) Deep learning. Nature 521:436–444
64. Patel YS, Misra R (2018) Performance comparison of deep VM workload prediction approaches for cloud. In: Prog Comput Anal Netw. Springer, pp 149–160
65. Gupta S, Dinesh DA (2018) Resource usage prediction of cloud workloads using deep bidirectional long short term memory networks. In: 11th IEEE Int Conf Adv Networks Telecommun Syst ANTS 2017 1–6
66. Calheiros RN, Ranjan R, Beloglazov A, De Rose CAF, Buyya R (2011) CloudSim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms. Softw Pract Exp 41:23–50

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH ("Springer Nature").

Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users ("Users"), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use ("Terms"). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.

These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.

We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.

While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval , sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.

These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.

To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.

Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.

If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

onlineservice@springernature.com